

Introduction to Docker

1. What is Docker?

Docker is an open-source platform that allows developers to package applications, along with all of their dependencies, into standardized units called containers. A container includes everything an application needs to run - code, runtime, system tools, libraries, and configuration files - so it behaves the same way regardless of where it is deployed.

Docker was first released in 2013 and quickly became the standard tool for containerization, changing how modern software is built, shipped, and deployed.

2. The Problem Docker Solves

Before containerization became common, developers frequently ran into the problem of "it works on my machine." An application might work perfectly on a developer's laptop but fail on a colleague's computer or in production, due to differences in operating systems, installed libraries, or configuration. Docker solves this by packaging an application and its entire environment into a single, portable unit that runs identically everywhere.

3. Containers and Images

Two of the most important concepts in Docker are images and containers.

- **Image:** A read-only template that contains the application code, dependencies, and instructions needed to run the application. Images are built once and can be reused.
- **Container:** A running instance of an image. Multiple containers can be started from the same image, each isolated from the others.

4. Dockerfile

A Dockerfile is a text file containing a series of instructions used to build a Docker image. Each instruction adds a layer to the image, such as installing dependencies or copying application code.

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

5. Docker Hub

Docker Hub is a cloud-based registry where Docker images can be stored, shared, and downloaded. It hosts thousands of official images for popular software such as Python, Node.js, PostgreSQL, and Nginx, as well as public and private images created by users and organizations.

6. Basic Docker Commands

<code>docker --version</code>	# Check installed Docker version
<code>docker images</code>	# List downloaded images
<code>docker ps</code>	# List running containers
<code>docker ps -a</code>	# List all containers, including stopped ones
<code>docker stop <container></code>	# Stop a running container
<code>docker rm <container></code>	# Remove a container
<code>docker rmi <image></code>	# Remove an image

7. Creating a Docker Image

Once a Dockerfile is written, an image can be built using the `docker build` command. The `-t` flag assigns a name and tag to the resulting image.

```
docker build -t my-python-app:1.0 .
```

8. Running a Container

A container is started from an image using the `docker run` command.

```
docker run my-python-app:1.0
```

Adding the `-d` flag runs the container in detached mode, meaning it runs in the background:

```
docker run -d my-python-app:1.0
```

9. Port Mapping

By default, a container's network ports are not accessible from outside the container. Port mapping connects a port on the host machine to a port inside the container, using the `-p` flag in the format `host_port:container_port`.

```
docker run -d -p 8000:8000 my-python-app:1.0
```

10. Volumes

By default, data created inside a container is lost when the container is removed. Volumes provide a way to persist data outside the container's filesystem, and can also be used to share files between the host machine and a container during development.

```
docker run -d -v $(pwd)/data:/app/data my-python-app:1.0
```

11. Environment Variables

Environment variables can be passed into a container at runtime using the `-e` flag. This is commonly used to configure settings such as database connection strings without hard-coding them into the image.

```
docker run -d -e DATABASE_URL=postgresql://db:5432/mydb my-python-app:1.0
```

12. Common Use Cases

- Ensuring applications run consistently across development, testing, and production.
- Packaging microservices so each service can be deployed and scaled independently.
- Simplifying onboarding by letting new developers run a full environment with one command.
- Running databases, message queues, and other infrastructure locally for development.
- Enabling continuous integration and continuous deployment (CI/CD) pipelines.

13. Conclusion

Docker has become a foundational tool in modern software development by solving the long-standing problem of inconsistent environments. By packaging applications into portable, isolated containers, Docker makes it easier to build, test, and deploy software reliably across different machines and environments. Understanding images, containers, Dockerfiles, and basic commands provides a solid starting point for working with containerized applications.